Bit-Exact Execution of a Connectome-Constrained Olfactory Spiking Neural Network on a Low-Cost FPGA

Draft, 25 October 2026

Abstract

> We show that a connectome-constrained recurrent spiking neural network (SNN) of the fruit fly runs completely and bit-exactly on a single low-cost FPGA. The subject is the publicly available model `flywire-olfactory-snn`: 52 inputs, 800 leaky integrate-and-fire neurons, 14,998 recurrent synapses taken from the FlyWire connectome, and 692 odorant classes from the DoOR database.
>
> From the PyTorch/Norse original we derive a numpy floating-point reference (logit deviation $< 10^{-5}$) and from it a fixed-point specification with 9-bit weights and Q8.7 neuron states. We implement it in Verilog on a Gowin GW2AR-18 (Tang Nano 20K board). The recurrent layer and the input weights reside in block RAM; the output layer resides in the in-package SDRAM.
>
> On the hardware, all 800 spike counts and all 692 logits match the reference bit for bit in 10 of 10 test samples. One classification with five-fold Monte Carlo averaging takes 1.72 M cycles, i.e. 31.8 ms at 54 MHz; we measured 66 ms per sample including UART transfer of all logits. The design uses 35 % of the LUTs, 21 % of the flip-flops and 42 of 46 block RAMs.
>
> Along the way we document properties of the published model that matter for any hardware implementation: the order of the LIF update in Norse 1.1.0, 46 % of synapse signs flipped against the connectome, and 32 % of weights that have effectively vanished. The variability introduced by the stochastic input encoding clearly exceeds the quantization error.

1 Introduction

The goal is a "fly nose" for the DeepRobotics Lite3 quadruped. Gas sensors provide measurements, a biologically constrained SNN classifies them in real time, and the robot reacts, for example with a source search modelled on insect chemotaxis. Compute hardware on a legged robot must be small, frugal and robust; an entry-level FPGA board meets these requirements if the network fits.

Connectome-constrained networks adopt the wiring of real neurons. With the complete connectome of the adult *Drosophila* brain (FlyWire, Dorkenwald et al. 2024), subgraphs such as the olfactory pathway from the antennal lobe to the Kenyon cells of the mushroom body can be used directly as the topology of a recurrent SNN. The model used here trains only the weight magnitudes on the fixed topology.

Hardware implementations of SNNs often target large neuromorphic chips or expensive FPGAs. We study the opposite: a single entry-level FPGA with 20,736 LUTs, 46 block RAMs and 64 Mbit of in-package SDRAM, built exclusively with open-source tools (Yosys, nextpnr-himbaechel, Apicula).

Contributions:

1. A floating-point reference verified against PyTorch/Norse and a bit-exact fixed-point specification of the model.
2. A single-chip architecture that executes the complete model including the 692-class readout.
3. An end-to-end verification chain down to the hardware with bit-exact agreement on the board.

4. Findings about the published model that matter for reuse and further hardware implementations.
5. A pipeline for connecting technical gas sensors, with retraining under a frozen connectome.

2 Model and Reference Implementation

The model `MaskedRecurrentLIFSNN` maps a 52-dimensional vector of odorant receptor responses to 692 odorant classes. Each sample is min-max normalized to rates r ∈ [0, 1]. In each of T = 20 time steps, Poisson spikes e ∈ {0, 1} are drawn; the input current is $W_{in} \cdot (e \odot r)$, i.e. spike times rate. The recurrent current from the previous step's spikes is added.

$$I_t = W_{in}\,(e_t \odot r) + W_{eff}\,s_{t-1}, \qquad W_{eff} = W_{rec} \odot M \odot S$$

M is the binary FlyWire adjacency (800 × 800, 14,998 edges, density 2.34 %), S the sign given by the transmitter of the presynaptic neuron. The neurons are Norse `LIFCell`s with default parameters: $dt \cdot \tau_{mem}^{-1} = 0.1$, $dt \cdot \tau_{syn}^{-1} = 0.2$, threshold 1, reset 0. The readout is linear in the mean spike rate; evaluation uses five-fold Monte Carlo averaging.

Floating-point reference. A numpy implementation reproduces the original, with `torch.bernoulli` replaced by externally supplied spikes. Against PyTorch 2.14 and Norse 1.1.0 the largest logit deviation is $5 \cdot 10^{-6}$ with 100 % identical argmax. The update order of Norse 1.1.0 is decisive: the input is added to the synaptic current first, and the membrane potential uses the new current.

$$i' = i + I_t, \quad v' = v + 0.1\,(i' - v), \quad i \leftarrow 0.8\,i', \quad z = [v' > 1], \quad v \leftarrow (1 - z)\,v'$$

Fixed-point specification. The hardware computes with integers only. The specification is itself an executable Python model against which all later stages are checked.

Quantity	Format
W_{in}, W_{eff}	9-bit signed, power-of-two scale 2^{-5} and 2^{-6}
W_{out}	8-bit, scale 2^{-5}
v, i	int16, Q8.7 with saturation (threshold 128)
Rate r	Q0.8, 0 to 256
Leak	$(K \cdot x + 2^{15}) \gg 16$ with $K_{mem} = 6554$, $K_{syn} = 13107$
Poisson spikes	xorshift32 per inference, $e = [x_{31\ldots24} < r]$
Readout	counts · W_{out} + b, factor 1/(20 · 5) folded into the weights

Q8.7 was chosen because Q5.10 produced more than 800,000 saturations of the synaptic current with the trained model. Because the readout is linear, the spike counts of all five Monte Carlo runs may be summed and the readout computed once; this is exactly equivalent to averaging the logits.

3 Hardware Architecture

The entire model runs on one GW2AR-18: connections and input weights reside in block RAM (BSRAM), the 541 KiB of the output layer in the in-package SDRAM. Thanks to the low density of 2.34 %, no partitioning across several chips is needed; an initially planned ring architecture over ten boards turned out to be unnecessary.

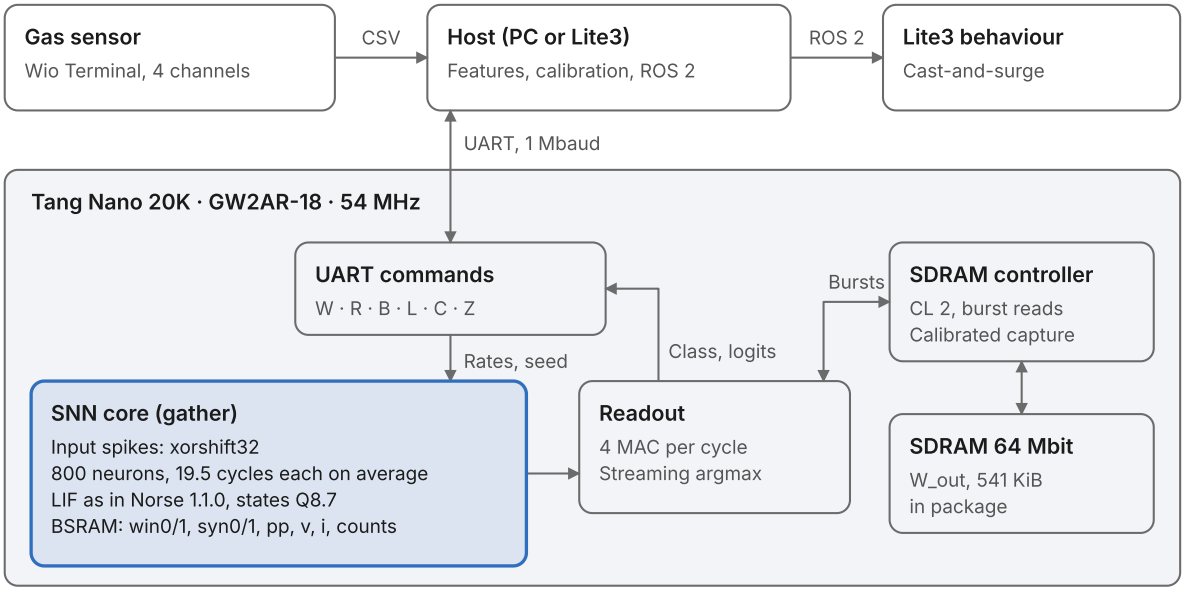


Figure 1: System overview. The host sends rates and a seed via UART. The core computes 100 time steps from BSRAM and hands the spike counts to the readout, which fetches W_{out} in bursts from the SDRAM and returns the class and logits.

Memory map. A synapse word holds a 9-bit weight and a source index. A 10-bit index does not fit next to a 9-bit weight in an 18-bit BSRAM word. The synapses are therefore split by source half (neurons 0–399 and 400–799) into two memories; a 9-bit index suffices and both halves are read in parallel.

Memory	Contents	Blocks
syn0, syn1	4,120 + 5,611 synapses, {source index 9 bit, weight 9 bit}	11
pp	801 list start pointers per target neuron	2
win0, win1	800 × 52 input weights, 4 readable per cycle	22
v, i, counts	states and spike counters	3
b_{out}, logit buffer	bias, response buffer	4
SDRAM	W_{out}, 692 × 200 words of 4 weights each	541 KiB

Compute core. A single time-multiplexed core processes the 800 neurons sequentially and gathers the inputs of each target neuron. Per neuron, the input path (13 cycles for 52 weights) and both synapse halves (one word per cycle each) run in parallel through a two-stage multiplier pipeline, followed by the LIF update in two cycles. The list ends of the next neuron are prefetched, so a neuron takes 19.5 cycles on average. Gathering instead of scattering avoids an accumulator memory with read-write hazards and makes the run time independent of the firing rate.

Readout and SDRAM. After 5 × 20 time steps the core reads one burst of 200 SDRAM words per class and performs four multiplications per cycle; the argmax is tracked in the data stream. The custom SDRAM controller uses CAS latency 2, one READ per cycle and a refresh every 600 cycles. The SDRAM clock is the inverted system clock.

Sample-phase calibration. The capture point of the read data can be set at run time in half-cycle steps: latency 2 to 7 cycles, optionally on the falling edge. After loading the weights, the host checks every capture point with real bursts and selects the centre of the error-free range (Section 5).

Interface. The host talks to the board via the board's USB UART at 1 Mbaud. Commands: W (write words to SDRAM, with checksum), R (single-word read), B (burst read), L (capture point), C (classify: seed and 52 rates, response: class and all logits), Z (spike counts).

4 Verification Methodology

Each stage is checked against the previous one with identical input spikes; from the fixed-point specification onwards the criterion is bit equality, not closeness.

1. **PyTorch/Norse → floating-point reference.** The original module is loaded with the published weights; `torch.bernoulli` returns the prescribed spikes. Criterion: logit deviation < 10^{-3}.
2. **Floating point → fixed point.** Identical spikes; we measure spike agreement, argmax agreement and value ranges. The latter determine the register widths in the RTL (input accumulator 20 bits, current 14 bits, readout 18 bits).
3. **Fixed point → RTL core (Verilator).** An export script generates memory images and test vectors: rates, seeds, the spike vector of every time step, counters and all logits. The testbench compares each of the 800 spikes in every step and every logit; the SDRAM is emulated as a data stream with random gaps.
4. **SDRAM controller.** A behavioural model checks protocol and timing (tRCD, tRP, tRFC, open rows, refresh interval) and returns read data with a realistic output delay.
5. **Complete system (Verilator).** The full top level with PLL bypass, bit-level UART and SDRAM model: loading, read-back, classification, logits and classes bit-exact.
6. **Hardware.** A host script loads W_{out} via UART, reads back samples, calibrates the capture phase and compares the class and all logits of each sample with the fixed-point reference. On a mismatch it reads the spike counters and thereby separates errors in the core from errors in the readout.

The Poisson spikes are produced on the chip and in the reference model by the same xorshift32 generator with the same seed. The stochastic part is thus also comparable bit for bit; the Monte Carlo statistics of the original (`torch.bernoulli`) are reproduced in distribution only, not bitwise.

5 Results

All stages of the chain are bit-exact or within tolerance, including the real hardware. Inputs are standard-normal random vectors as in the model's example; DoOR test data with labels were not available.

Verification.

Stage	Scope	Result
Float reference vs. PyTorch/Norse 1.1.0	64 samples × 5 runs	max. logit deviation $5.3 \cdot 10^{-6}$, argmax 100 % identical
Fixed point vs. float, identical spikes	300 samples × 5 runs	spikes 98.5 % identical, argmax 96.3 % identical, no saturation
RTL core (Verilator) vs. fixed point	20 samples, 2,000 time steps	0 spike, 0 logit, 0 class errors (13,840 logits)
SDRAM controller vs. model	4,096 writes, 3,000 reads	0 data, 0 protocol errors, refresh interval ≤ 779 cycles
Complete system (Verilator, UART, SDRAM)	2 samples + memory test	all logits and classes bit-exact
Board, core with counter output, 27 MHz	10 samples	8,000 of 8,000 spike counts bit-exact
Board, full design, 54 MHz	10 samples	6,920 of 6,920 logits and 10 of 10 classes bit-exact

Run time. The first core version needed 3.04 M cycles per classification (112 ms at 27 MHz). With parallel synapse halves, four input weights per cycle, pipelining and pointer prefetch it needs 1.72 M cycles, i.e. 31.8 ms at 54 MHz. On the board we measured 66 ms per sample, including the UART response with all 692 logits (2,780 bytes at 1 Mbaud). Loading the output weights takes 6.0 s once.

Resources (nextpnr, full design):

Resource	Used	Available
LUT4	7,265 (35 %)	20,736
Flip-flops	≈ 3,300 (21 %)	15,552
BSRAM	42 (91 %)	46
Multipliers	6 × 18×18, 4 × 9×9	48 / 96
Fmax	58.2 MHz	54 MHz used

Quantization versus encoding noise. Two floating-point runs with different seeds yield the same class in only 71.3 % of cases. Measured against the mean over 50 runs, floating point and 8-bit fixed point with five runs each both reach 82 %; 12-bit weights change nothing (81.7 %). The quantization error thus disappears in the Monte Carlo noise of the input encoding. More runs help more than more bits.

SDRAM capture window. Single-word accesses were correct on the board at every setting, bursts were not: a single access hides a wrong capture point because the data bus still holds the value afterwards. Calibration with bursts revealed exactly one error-free capture point:

Capture after READ	1.5 cycles	2.0 cycles	2.5 cycles (falling edge)	3.0 to 7.0 cycles
Wrong burst words out of 200	200	161	0	198

The valid data window lies about half a cycle earlier than assumed in the simulation model. Without half-cycle steps it could not have been hit.

6 Discussion

Findings about the published model. The implementation required a close reading of code and weights. It revealed properties that the model description does not mention and that affect any reuse:

- The model card states 500 classes; configuration and weights contain 692.
- Norse 1.1.0 adds the input before the membrane update. A core using the order of older versions is off by one time step (measured: logit deviation $7 \cdot 10^{-2}$, 16 % differing classes).
- The model card describes learned magnitudes on fixed signs. The code, however, does not constrain the recurrent parameter to ≥ 0; after training, 6,905 of 14,998 signs (46 %) are flipped against the transmitter annotation, and only 105 of 800 neurons act uniformly excitatory or inhibitory. Only the topology is therefore certainly biologically prescribed.
- 4,773 weights (32 %) are effectively zero ($|w| < 10^{-30}$), presumably due to weight decay. After 9-bit quantization 9,731 synapses remain.
- The per-sample min-max normalization discards the absolute concentration and amplifies sensor noise in clean air to the full value range.

Architecture choice. Given the real weights, the network fits on one chip; compute is not the bottleneck, memory is (91 % of BSRAMs). Additional boards are therefore best used as independent copies: five boards compute the five Monte Carlo runs in parallel, or one board serves each of two sensor heads. This is exactly equivalent be-

cause the readout is linear in the spike counts. A cluster with spike exchange pays off only for larger connectome subgraphs.

Limitations.

- We verified bit-exactness against the reference, not classification accuracy on DoOR test data; the inputs were random vectors. The test accuracy reported by the model author (47.6 % on 692 classes) was not re-measured.
- The hardware tests comprise 10 samples per configuration on one board at room temperature.
- The error-free SDRAM capture window is only half a cycle wide. Calibration at start-up absorbs variation; for operation in a warm robot a finer phase setting via the second PLL output is advisable.
- The output weights are loaded via UART after each power-up (6 s).

7 Outlook: Sensors and Robot

Technical gas sensors do not deliver fly receptor responses. We therefore retrain the input and output layers on our own sensor recordings and keep the connectome (topology and recurrent weights) frozen. The pipeline is implemented but so far tested only with synthetic data.

- **Sensor:** Grove multichannel gas sensor (NO_2, CO, C_2H_5OH, VOC) on a Wio Terminal, read out through the existing firmware.
- **Features:** 10-s windows resampled to 10 Hz, logarithmic relative to a tracked baseline; 13 features per channel give the 52 model inputs. An intensity derived from the deviation from baseline gates out clean air and replaces the concentration lost to min-max normalization.
- **Retraining:** surrogate-gradient training with the original module and frozen recurrent weights; additional evaluation with the bit-exact fixed-point model.
- **Synthetic test (5 classes):** validation accuracy 95.5 % (float) and 95.8 % (fixed point); the retrained model runs bit-exactly in RTL simulation including SDRAM and UART. Synthetic data are considerably easier than real recordings.
- **Live operation:** a host program connects sensor, FPGA and optionally ROS 2 (`/olfaction/odor`, `/olfaction/intensity`).

On the Lite3, a cast-and-surge behaviour modelled on flying insects is to use the class and the intensity: move up the gradient while the odour increases, and cast sideways when it is lost. Open steps are real recordings of several odours over several days, a comparison with an equally sized random network as a control for the benefit of the fly topology, and sensors with more channels, for example BME688 with heater profiles.

8 Conclusion and Availability

A connectome-constrained olfactory SNN with 800 neurons and 692 classes can be executed completely and bit-exactly on a single entry-level FPGA, in 31.8 ms per classification and with open tools only. Key ingredients were an executable fixed-point specification as a shared reference, a verification chain with bit equality as the criterion, and an SDRAM capture point that can be calibrated at run time. The findings about the model show that for connectome-constrained networks one should check how much biological structure actually survives training.

Code. Reference models, export tools, Verilog, testbenches, build scripts and host programs are in the `snn-fpga/` directory of the repository drhenrich/artificial-nose, branch `snn-fpga-konzept`.

Data. Model weights and memory images derived from them are not included; they are generated locally from the published model. The licence of the model weights is not stated on the model page; the underlying DoOR data are licensed CC BY-SA 4.0, and FlyWire's terms of use apply to FlyWire data.

References

All entries were checked against publisher, PubMed/PMC, NSF, Zenodo and project records on 25 October 2026; entry 3 is the model page as provided by the model author.

1. Dorkenwald, S. et al. (2024). Neuronal wiring diagram of an adult brain. *Nature* 634(8032), 124–138. doi:10.1038/s41586-024-07558-y
2. Münch, D. & Galizia, C. G. (2016). DoOR 2.0 – Comprehensive mapping of *Drosophila melanogaster* odorant responses. *Scientific Reports* 6, 21841. doi:10.1038/srep21841
3. Vick-MIRE. flywire-olfactory-snn (MaskedRecurrentLIFSNN). Hugging Face. huggingface.co/Vick-MIRE/flywire-olfactory-snn
4. Pehle, C. & Pedersen, J. E. (2021). Norse – A deep learning library for spiking neural networks (version 0.0.7). Zenodo. doi:10.5281/zenodo.4422025
5. Zenke, F. & Ganguli, S. (2018). SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation* 30(6), 1514–1541. doi:10.1162/neco_a_01086
6. Neftci, E. O., Mostafa, H. & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine* 36(6), 51–63. doi:10.1109/MSP.2019.2931595
7. Wolf, C. X. (formerly C. Wolf). Yosys Open SYnthesis Suite. yosyshq.net/yosys
8. YosysHQ. nextpnr – portable FPGA place and route tool (himbaechel architecture, Gowin target). github.com/YosysHQ/nextpnr
9. Project Apicula. Documentation and open-source tools for the Gowin FPGA bitstream format. github.com/YosysHQ/apicula